



PICCIONE TCG — Documento di Design Completo

Versione: 0.1 — Bozza Concettuale

Progetto: Bot Discord "Piccione" — Modulo TCG

Autore: Conversazione collaborativa (Utente + Gemini AI)

Nota: Questo documento raccoglie e sistematizza tutte le idee emerse durante la sessione di design.

1. VISIONE GENERALE

Il **Piccione TCG** è un Trading Card Game competitivo integrato direttamente nel bot Discord "Piccione" — il bot già attivo per la gestione delle news nei thread forum del server. L'obiettivo è trasformare un bot di utilità in un **hub di intrattenimento e community** che generi engagement quotidiano attraverso:

- Collezionismo e apertura pacchetti
- Costruzione di mazzi personalizzati
- Sfide 1v1 e multiplayer (Commander)
- Missioni legate all'attività del server
- Scambi di carte tra giocatori
- Tornei con ricompense esclusive

Il tema è interamente **a base di piccioni e meme**. Ogni carta, ogni meccanica, ogni nome riflette il mondo del piccione. Questo crea un'identità unica, memorabile e comica che favorisce la viralità interna al server.

2. STRUTTURA DEL GIOCO

2.1 Obiettivo

Ridurre i **Punti Vita (PV)** dell'avversario a 0. Nel formato Commander (multiplayer), eliminare tutti gli avversari o soddisfare condizioni di vittoria alternative.

2.2 Punti Vita

Formato	PV Iniziali
Duello Standard (1v1)	20 PV
Commander (Multiplayer)	60 PV

I 60 PV nel formato Commander bilanciano il sistema di risorse a dado, che tende ad essere esplosivo nelle fasi avanzate della partita. Questo "cuscinetto" permette di vedere combo e strategie complesse prima della fine della partita.

3. SISTEMA DI RISORSE — "MOLLICHE E SPEZIE"

Questa è la **killer feature** del gioco. Niente mana fisso, niente terreni. Le risorse sono generate da dadi (RNG) e possono essere "condite" con le **Spezie** (le fazioni del gioco).

3.1 Le Molliche (La Risorsa Base)

All'inizio di ogni turno, il bot lancia automaticamente un numero di **dadi D4** che cresce col progredire della partita. Il totale dei dadi diventa la riserva di "Molliche" spendibili per quel turno.

Progressione dei Dadi (bozza):

Turno	Dadi Lanciati	Range Risorse
1–2	1d4	1–4
3–4	2d4	2–8
5–6	3d4	3–12
7–8	4d4	4–16
9–10	5d4	5–20
...	+1d4 ogni 2 turni	...

Nota sul bilanciamento: È consigliato aggiungere una "Mollica Garantita" come offset minimo ($1 + [\text{risultato dadi}]$) per evitare turni dove il giocatore non può giocare nulla per sfortuna pura. Da testare in playtest.

Nessun cap attuale: La progressione dei dadi non ha un massimale definito. Questo renderà le fasi tardive delle partite molto esplosive. Le carte **Balcone** e **Virata** potranno regolare questo aspetto indirettamente.

3.2 Le Spezie (La Fazione/Colore)

Ogni dado ottenuto è "neutro". Nella **Fase di Condimento** (prima della Main Phase), il giocatore assegna ogni dado a una **Spezia**. Le carte hanno un costo in Spezie specifiche; si può spendere solo la Spezia corretta per giocarle.

Esempio: Se peschi 3 dadi (valori: 1, 3, 2 = 6 Molliche totali), puoi assegnare 2 a Paprica e 1 a Origano. Potrai giocare carte da Paprica fino a 4 Molliche e carte da Origano fino a 2.

Le Spezie Definite:

Spezia	Colore MTG equivalente	Archetipo / Tema
 Paprica	Rosso	Danno diretto, aggressività, rush
 Origano	Blu/Verde	Controllo, protezione, generazione risorse
 Pepe Nero	Nero	Sacrificio, rimozione, effetti di morte
 Sale	Bianco	Buff di gruppo, armatura, creazione token

Potranno essere aggiunte nuove Spezie nelle espansioni future.

3.3 L'Identità del Comandante e le Spezie

Il **Comandante** (carta leggendaria) definisce a quali Spezie il giocatore ha accesso:

- **Comandante Monospezia** → Solo 1 Spezia disponibile (mazzo più coeso, ma meno flessibile)
- **Comandante Multispezia** → Accesso a 2 o più Spezie (rare, molto ambite)
- **Comandante Gourmet** (leggendario raro) → Accesso a tutte le Spezie (combinazioni infinite, ma carte con costi più alti)

Questo sistema risolve alla radice il problema del "mana screw" tipico di MTG: ogni dado è sempre utile, basta decidere come condirlo.

3.4 Le Portaspezie (Acceleratori di Risorse)

Le carte di tipo **Portaspezie** modificano il sistema dei dadi:

- Possono permettere di **lanciare un dado extra** per turno
- Permettono di **ri-lanciare** un dado con risultato basso
- Convertono una Spezia in un'altra
- Fissano un dado al valore massimo (4) per un turno

4. TIPOLOGIE DI CARTE

Tutte le carte hanno un **nome tematico "piccionico"** e appartengono a uno dei seguenti tipi:

4.1 **Piccione (Creatura)**

Equivalente: Creatura (MTG) / Mostro (YGO)

- Hanno statistiche di **ATK** e **DEF**
- Possono attaccare e bloccare
- Hanno abilità passive o attivate

- **Tutti i piccioni appartengono alla "tribale Piccione"**, il che rende quasi ogni carta sinergica con le altre
- I token generati da effetti sono anch'essi Piccioni (a meno che specificato)

Meccanica di posizione in campo:

- Il campo ha **5 slot per lato**
- Le carte in posizione **Fronte** proteggono quelle in posizione **Retro**
- Le carte con la keyword **Volò** possono bypassare la linea frontale

4.2 🏠 Balcone (Terreno Globale)

Equivalente: Terreno (YGO) / Incantesimo Globale (MTG)

- Una volta giocato, rimane in campo finché non viene distrutto
- Modifica le regole per **entrambi i giocatori** (o solo per chi lo ha giocato, a seconda dell'effetto)
- Il sistema mantiene una lista `active_balconies` che viene verificata ad ogni calcolo
- Più Balconi possono coesistere se non si contraddicono

Esempio: `Piazza Affollata` — Tutti i Piccioni in campo guadagnano +1/+0.

4.3 🗡️ Portaspezie (Acceleratore di Risorse)

Equivalente: Mana Rock (MTG) / Magie continue (YGO)

- Carte permanenti che modificano la generazione di Molliche
- Rimangono in campo e hanno un effetto continuo sui dadi
- Possono essere distrutte come qualsiasi permanente

4.4 ✨ Picchiata (Istantaneo)

Equivalente: Istantaneo (MTG)

- Giocabile in **qualsiasi momento**, anche durante il turno dell'avversario
- Il sistema prevede un meccanismo di **Interrupt**: quando un giocatore annuncia un'azione, gli avversari hanno una finestra per rispondere con Picchiate
- Il bot mette il gioco in stato di "attesa" (`pause`) e notifica i giocatori

4.5 🌀 Virata (Stregoneria)

Equivalente: Stregoneria (MTG) / Magia normale (YGO)

- Giocabile **solo durante il proprio turno**, nella Main Phase
- Effetti più potenti rispetto alle Picchiate in cambio di questa limitazione
- Possono includere draw, board wipe, potenziamenti massivi, ecc.

4.6 🍷 Token

Equivalente: Token (MTG/YGO)

- Entità create dagli effetti di altre carte
- Non appartengono al mazzo (vengono generate dal sistema)
- Hanno template predefiniti: `Piccione Comune (1/2)`, `Piccione Guerriero (2/2)`, `Uovo (0/1)`, ecc.
- Spariscono se lasciati fuori dal campo (dipende dal design della carta che li genera)
- Sono **sempre Piccioni**, quindi interagiscono con tutte le sinergie tribali

5. KEYWORDS (PAROLE CHIAVE)

Le keywords permettono di creare effetti complessi con descrizioni sintetiche:

Keyword	Effetto
Volo	Può attaccare direttamente l'avversario (ignora i Piccioni in posizione Fronte)
Piccionaia	Se questa carta è in campo, tutte le altre tue carte guadagnano +1 HP
Muta	Si trasforma in un'altra carta dopo X turni in campo
Rumore	L'avversario non può usare abilità attivate per 1 turno
Rush	Può attaccare il turno stesso in cui viene giocata
Volo Ristretto	Come Volo, ma solo contro target con HP ≤ 2
Sacrificio	Può essere sacrificata per attivare un effetto
Nidificazione	Genera un token all'inizio di ogni tuo turno

L'elenco si espande con il design delle carte. L'obiettivo è avere effetti modulari: `OnPlay`, `OnDeath`, `StartOfTurn`, `OnAttack`, *con target* `Self`, `Enemy`, `AllBoard`, `Random`.

6. FASI DEL TURNO

1. DRAW PHASE

→ Il giocatore pesca 1 carta dal mazzo
2. RESOURCE PHASE

→ Il bot lancia automaticamente i dadi D4
3. SPICE PHASE

→ Il giocatore assegna i dadi alle Spezie ("Fase di Condimento")
4. MAIN PHASE

→ Il giocatore gioca carte (Piccioni, Balconi, Portaspezie, Virate) e può attaccare
5. ATTACK PHASE

→ Dichiarazione degli attaccanti e dei bloccanti
6. RESPONSE WINDOW

→ Gli avversari possono rispondere con Picchiate
7. END PHASE

→ Reset delle abilità usate, fine turno

7. CARTE INIZIALI (Esempi di Design)

Le prime carte definite durante la sessione di brainstorming:

Piccione (Comune)

"Non ha abilità, ma ha il potere di essere ovunque."

- **Tipo:** Piccione — Creatura Comune
 - **Costo:** 1 Mollica (qualsiasi Spezia)
 - **ATK/HP:** 1 / 2
 - **Effetto:** Nessuno
 - **Note di Design:** La carta più comune di tutte. Alta probabilità nei pacchetti. Adatta per sfide "chi ha più Piccioni base" e challenge di community.
-

Spennata (Comune)

- **Tipo:** Picchiata — Istantaneo Comune
 - **Costo:** 1 Mollica (qualsiasi)
 - **Effetto:** Infligge 2 danni a una creatura bersaglio.
 - **Note di Design:** Rimozione economica. Uccide i Piccioni base. Ottima come risposta.
-

Covata (Non Comune)

- **Tipo:** Balcone — Permanente Non Comune
 - **Costo:** 3 Molliche
 - **Effetto:** All'inizio del tuo turno, genera un token Piccione (1/2).
 - **Note di Design:** Carta di controllo del board. Se non rimossa rapidamente, il vantaggio numerico diventa insostenibile per l'avversario.
-

8. FORMATO COMMANDER (EDH "Piccionico")

8.1 Il Piccione Comandante

- Ogni giocatore sceglie un **Piccione Leggendaro** come Comandante prima della partita
- Il Comandante parte in una zona separata: il **"Posatoio"**
- Può essere giocato in qualsiasi momento spendendo il suo costo in Molliche
- Il Comandante **entra in campo come qualsiasi altra carta**: deve essere giocato, non è automaticamente attivo

8.2 La Tassa del Piccione

- Ogni volta che il Comandante viene sconfitto e torna al Posatoio, il suo **costo aumenta di 1 Mollica**
- Questo previene lo spam infinito e crea una tensione strategica attorno alla sua sopravvivenza
- L'interfaccia mostra sempre il contatore: `Tassa: +0`, `Tassa: +1`, ecc.

8.3 Identità di Fazione

- Il mazzo può contenere **solo carte della/e Spezia/e del Comandante**
- Comandanti con più Spezie permettono mazzi più vari ma richiedono gestione più complessa

8.4 Multiplayer (4+ Giocatori)

- Il turno segue l'ordine cronologico di ingresso nella partita (gestito dal bot)
- Il bot crea un **Thread Discord dedicato** alla partita per la cronaca pubblica
- Gli spettatori possono seguire il thread, commentare e creare engagement organico
- **Engagement politico:** I giocatori trattano nel thread ("Non attaccarmi e ti aiuto con X!") → massima attività del server

8.5 Visuale 4-Giocatori

L'interfaccia passa da una visuale verticale (1v1) a una **a croce o a slot rotanti** (multiplayer). I quattro giocatori occupano i quattro lati del campo di gioco visualizzato nell'Attività Discord.

9. ARCHITETTURA TECNICA (SISTEMA IBRIDO)

La scelta fondamentale dell'architettura: il gioco è diviso tra **Bot (Notaio)** e **Discord Activity (Motore di Gioco)**.

9.1 Il Bot Piccione — Il "Notaio"

Il bot rimane il **custode dell'economia e dell'integrità del gioco**. Non gestisce la logica in-game frame per frame, ma valida e persiste tutti i dati importanti.

Responsabilità:

- Database: carte, inventari, mazzi, statistiche, missioni
- Distribuzione pacchetti (con RNG server-side)
- Validazione delle mosse dichiarate dall'Activity (`Ha davvero questa carta? Ha abbastanza Molliche?`)
- Distribuzione ricompense a fine partita
- Matchmaking e creazione dei Thread di partita
- Gestione degli scambi tra giocatori (con transazioni atomiche)
- Pubblicazione degli eventi di sistema nel server (annunci tornei, eventi, ecc.)

9.2 La Discord Activity — Il "Palcoscenico"

Una **web-app** (React + Vite) ospitata su server dedicato e caricata come Discord Activity (IFrame). Gestisce tutto il rendering e la logica di gameplay frame-per-frame.

Stack tecnologico consigliato:

- **Framework:** React + Vite
- **Animazioni:** Framer Motion (carte, attacchi, shake screen)
- **Animazioni asset:** Lottie Files (esplosioni, effetti spezia, apertura pacchetti)
- **Ricerca locale:** Fuse.js o FlexSearch (per il filtro carte nel Deck Builder)
- **Comunicazione:** Discord Embedded App SDK + WebSocket verso il bot
- **Fisica dadi:** Three.js o Matter.js (opzionale, per dadi 3D)

Responsabilità:

- Rendering del campo di gioco (carte, slot, HP)
- Animazioni di attacco, difesa, effetti delle carte
- Lancio dei dadi con fisica (visivamente)
- Fase di Condimento (drag dei dadi sulle Spezie)
- Animazione apertura pacchetti
- Deck Builder con drag & drop
- Visualizzazione del "Piccionario" (Pokédex delle carte)

9.3 Flusso di una Partita (Step-by-Step)

```
1. Utente usa /sfida nel canale Discord
2. Il Bot crea una "stanza virtuale" e invia il link all'Activity
3. L'Activity chiede al Bot: "Dammi i mazzi dei giocatori X, Y, Z, W"
4. Il Bot risponde con i JSON delle carte
5. I giocatori giocano nell'Activity (logica client-side)
6. Per ogni mossa: Activity → Bot (validazione) → Activity (conferma + animazione)
7. A fine partita: Activity invia "Referto" → Bot distribuisce ricompense
8. Il Bot annuncia il vincitore nel Thread con un riassunto epico della partita
```

9.4 GameState Object (Contratto di Comunicazione)

```
{
  "match_id": "uuid-partita",
  "format": "commander",
  "players": [
    {
      "id": "discord_user_id",
      "hp": 60,
      "commander": { "card_id": 42, "tax": 0, "in_play": false },
      "field": [null, "card_obj", null, "card_obj", null],
      "hand": ["card_id_1", "card_id_2"],
      "spice_pool": { "paprica": 3, "origano": 2, "pepe_nero": 0, "sale": 1 },
      "dice_roll": [3, 1, 4, 2]
    }
  ],
  "active_balconies": ["card_obj_1"],
  "turn": "discord_user_id_1",
  "phase": "spice_phase",
  "turn_number": 5
}
```

9.5 Evento di Mossa (API Bot ↔ Activity)

```
{
  "action": "PLAY_CARD",
  "player_id": "discord_user_id",
  "card_id": 101,
  "target": "discord_user_id_2",
  "spice_used": { "paprica": 2 }
}
```

10. INTELLIGENZA ARTIFICIALE (NPC)

Per permettere sfide contro il bot (e per le missioni), ogni NPC ha una logica decisionale basata su una **Utility Function**.

10.1 Formula di Valutazione

$$\text{Score}(\text{carta}) = (\text{Potere} \times W1) + (\text{Sinergia} \times W2) - (\text{Costo} \times W3)$$

I Pesi (**W1**, **W2**, **W3**) cambiano in base alla personalità dell'NPC e alla situazione in campo (HP bassi → più peso alla difesa).

10.2 Livelli di Difficoltà

Livello	Algoritmo	Descrizione
Facile	Randomizzatore pesato	Mosse casuali, 20% probabilità di scegliere l'ottimale
Medio	Greedy	Sceglie sempre la mossa che massimizza il vantaggio immediato
Difficile	Mini-Minimax (2 turni)	Simula le prossime 2 mosse e sceglie la sequenza migliore

10.3 Personalità degli NPC

Ogni NPC ha un archetipo che ne definisce i pesi:

Archetipo	Stile	Pesi Elevati
L'Aggressivo	Rush, danno diretto	ATK alto, Rush, Paprica
Il Controllore	Difesa e rimozioni	Rimozioni, Balconi, Origano
Il Combo Player	Combo e sinergie	Synergy tags comuni, Token
Il Meme Lord	Caotico, imprevedibile	Random, spesso utilizza Piccioni base

10.4 Implementazione

```
def decide_move(hand, board, personality_weights):
    possible_moves = get_legal_moves(hand, board)
    best_move = None
    best_score = float('-inf')

    for move in possible_moves:
        score = evaluate_board_after(move, personality_weights)
        if score > best_score:
            best_score = score
            best_move = move

    return best_move
```

11. ECONOMIA DI GIOCO (LOOP DI ENGAGEMENT)

L'economia è il motore che tiene i giocatori attivi ogni giorno.

11.1 Valute

Valuta	Come si ottiene	Come si spende
 Piume	Smantellamento duplicati, missioni	Crafting carte specifiche, scambi
 Molliche Extra	Tornei, sfide NPC, eventi	Pacchetti aggiuntivi
 Gettoni Torneo	Solo dai tornei	Pacchetti esclusivi, skin carte

11.2 Pacchetti

- **Pacchetto Giornaliero:** 1 gratuito ogni 24h — garantisce l'apertura quotidiana
- **Pacchetto Standard:** acquistabile con Piume/Molliche Extra
- **Pacchetto Leggendaro:** solo da tornei o eventi speciali
- **Sistema Streak:** Apri il pacchetto 7 giorni di fila → l'ottavo giorno hai rarità garantita superiore

11.3 Smantellamento (Dusting)

Rarità	Piume ottenute
Comune	1
Non Comune	3
Rara	10
Leggendaria	50

11.4 Missioni (Fonti di Valuta)

Le missioni sono legate all'attività del server, integrando il modulo news con il TCG:

- **Missioni Giornaliere:** Vinci 1 partita, pesca 3 carte specifiche, ecc.
- **Missioni del Server:** Commenta una news nel thread → +10 Piume
- **Missioni Sociali:** Completa uno scambio con un altro giocatore → +Molliche Extra
- **Missioni Torneo:** Piazza nei top 3 → Pacchetto Leggendaro
- **Bounty:** Il bot mette una "taglia" su chi ha vinto troppe partite. Chi lo batte riceve ricompense extra.

11.5 Scambi tra Giocatori

Il mercato delle carte è il **motore sociale** del gioco.

Funzionamento:

1. Il giocatore A propone lo scambio tramite comando `/scambia`
2. Il bot pubblica l'annuncio nel canale `#mercantino`
3. Il giocatore B accetta
4. Il bot **congela** entrambe le carte, verifica la disponibilità, e le scambia **atomicamente**

5. L'operazione è una transazione SQL — se fallisce a metà, nessuna carta si perde

Safe Trade garantito: nessuna possibilità di truffa perché il bot gestisce entrambi i lati simultaneamente.

11.6 Il "Piccionario" (Database Collezione)

- Elenco di tutte le carte esistenti nel gioco
- L'utente vede quali ha e quali gli mancano
- Organizzato per **Set** (es. Set Base, Set Città, Set Laboratorio, ecc.)
- **Ricompensa completamento Set:** Skin avatar, titolo speciale nel server, carta esclusiva
- **Vetrina profilo:** Ogni utente può esporre le sue 3 carte più rare

12. DECK BUILDER (NELL'ACTIVITY)

Il Deck Builder è integrato nell'Activity Discord per un'esperienza fluida e visivamente appagante.

12.1 Layout "Split-View"



12.2 Funzionalità Chiave

- **Drag & Drop:** Trascina la carta nel mazzo con feedback visivo/sonoro
- **Quick-Add:** Doppio click per aggiungere rapidamente (mobile-friendly)
- **Test Hand:** Simula una mano iniziale di 7 carte casuali dal mazzo
- **Deck Share:** Pubblica il mazzo nel thread Discord con un link cliccabile
- **Clone Mazzo:** Un altro giocatore può clonare il mazzo altrui come base di partenza
- **Validazione in tempo reale:** Il sistema avvisa se stai violando le regole di fazione del Comandante

12.3 Ottimizzazione Tecnica

- **Virtualizzazione lista:** Le immagini vengono caricate solo durante lo scroll (evita lag con 500+ carte)

- **Ricerca locale:** Fuse.js processa il filtro sul device dell'utente senza query al bot ad ogni keystroke

13. INTERFACCIA DI GIOCO (UI/UX)

13.1 Widget del Giocatore (In-Game)

Posizionato in basso al centro dello schermo:

- **Avatar Discord** con bordo colorato in base alla Spezia principale del mazzo
- **Barra HP circolare/lineare** che cambia colore: Verde → Giallo → Rosso
- **Contatore Molliche** con dado animato (mostra il tipo di dado e il risultato)
- **Contatore Tassa Comandante** (Tassa: +0)
- **Indicatore Comandante:** Icona di protezione se in campo, icona in cooldown se nel Posatoio

13.2 Effetti Visivi Dinamici

Evento	Effetto UI
Danno subito	Shaking screen + flash rosso ai bordi
HP < 20	Bordi dello schermo pulsanti rossi
Carta leggendaria giocata	Alone dorato animato + sound
Danno alto	Tremolio intenso dello schermo
Vittoria	Animazione coriandoli piccioni
Lancio dadi	Fisica 3D opzionale (Three.js)
Apertura pacchetto	Animazione busta che si strappa + carte che escono con rarità highlight

13.3 Il Campo da Gioco

```
[
  [AVVERSARIO - 60 HP]
  [slot][slot][slot][slot][slot] ← campo nemico
  _____
  [slot][slot][slot][slot][slot] ← campo giocatore
  [TU - 60 HP]
  [Dadi] [Spezie] [Mano] [Piazzati]
]
```

14. DATABASE (STRUTTURA CONSIGLIATA)

```
-- Utenti e carte
Users          (id, discord_id, piume, molliche_extra, gettoni_torneo, streak_days)
Cards          (id, name, type, cost, atk, hp, rarity, spezia, keywords[], effects[])
UserInventory  (user_id, card_id, quantity)

-- Mazzi
Decks          (id, user_id, name, commander_card_id, created_at)
DeckCards      (deck_id, card_id, quantity)

-- Partite
Matches        (id, format, status, winner_id, created_at, ended_at, thread_id)
MatchPlayers   (match_id, user_id, hp_final, commander_tax_final)
MatchEvents    (id, match_id, turn, event_type, data_json, timestamp)

-- Economia
Trades         (id, proposer_id, receiver_id, status, created_at)
TradeItems     (trade_id, owner_id, card_id, quantity)
PacketLogs     (id, user_id, pack_type, cards_received[], opened_at)

-- Missioni
Missions       (id, type, description, reward_type, reward_amount, refresh_period)
UserMissions   (user_id, mission_id, progress, completed_at)
```

Fondamentale: Tutti gli scambi e gli smantellamenti devono essere operazioni **atomiche** con transazioni SQL per prevenire perdite di carte in caso di crash.

15. SISTEMA DI RARITÀ E DROP RATE

Rarità	Simbolo	Drop Rate (bozza)	Piume Smantellamento
Comune		~60%	1
Non Comune		~25%	3
Rara		~10%	10
Epica		~4%	25
Leggendaria		~1%	50
Leggendaria Meme		<0.1%	150

Rarità "Leggendario Meme": Carte speciali legate a eventi del server (es. se viene postata una news con un piccione reale nel canale news, quella carta ottiene un buff temporaneo o viene aggiunta al pool come evento limitato).

16. ROADMAP DI IMPLEMENTAZIONE

Fase 1 — Backend Core (Bot Piccione)

- ☐ Schema database SQL (Users, Cards, Inventory, Decks)
- ☐ Sistema di apertura pacchetti con RNG server-side
- ☐ Comandi Discord: `/mieecarte`, `/pacchetto`, `/profilo`, `/piuma`
- ☐ Sistema di smantellamento duplicati
- ☐ Sistema di scambio Safe Trade

Fase 2 — Game Engine (Activity - Frontend)

- ☐ Setup React + Vite + Discord Embedded App SDK
- ☐ Rendering del campo di gioco base (1v1)
- ☐ Sistema di gestione del GameState lato client
- ☐ Fasi del turno con logica di validazione locale
- ☐ Comunicazione WebSocket con il bot per validazione mosse

Fase 3 — Deck Builder (Activity)

- ☐ Layout Split-View con drag & drop
- ☐ Filtro locale con Fuse.js
- ☐ Test Hand simulator
- ☐ Salvataggio mazzo → bot → database
- ☐ Deck Share nel thread

Fase 4 — Animazioni e UX (Activity)

- ☐ Animazioni Framer Motion (carte, attacchi, shake)
- ☐ Apertura pacchetti animata (Lottie Files)
- ☐ Widget giocatore con HP bar e dado animato
- ☐ Effetti visivi per rarità (alone dorato, ecc.)

Fase 5 — Formato Commander (Bot + Activity)

- ☐ Gestione della zona Posatoio
- ☐ Sistema di Commander Tax
- ☐ Supporto 4 giocatori (array PlayerIDs)
- ☐ Layout UI a croce per multiplayer
- ☐ Thread Discord per cronaca partita

Fase 6 — Missioni e Engagement

- ☐ Sistema missioni giornalieri
- ☐ Integrazione con thread news (leggi news → ricompensa)
- ☐ Pacchetto giornaliero con sistema Streak
- ☐ Sistema Bounty (taglie sui giocatori forti)
- ☐ Annunci eventi mensili nel server

Fase 7 — NPC e AI

- ☐ Game Manager class (Python) isolata dal bot Discord
- ☐ Greedy Algorithm (difficoltà media) come prima implementazione
- ☐ Personalità NPC con pesi configurabili
- ☐ Logging delle mosse NPC per affinamento

Fase 8 — Espansioni e Contenuti

- ☐ Nuove carte e Set tematici
- ☐ Nuove Spezie (fazioni)
- ☐ Sistema tornei strutturati
- ☐ Skin e cosmetici per carte/avatar
- ☐ Piccionario con ricompense completamento Set

17. CONSIGLI GENERALI PER L'IMPLEMENTAZIONE

1. **Inizia dal backend:** Definisci lo schema del database e i comandi base prima di toccare il frontend. Il database è la colonna vertebrale di tutto.
2. **"Contratto prima del codice":** Prima di implementare la comunicazione Bot↔Activity, definisci esattamente il formato JSON di ogni messaggio scambiato. Questo evita riscritture costose.
3. **Usa transazioni atomiche sempre:** Ogni operazione che modifica l'inventario (scambio, smantellamento, apertura pacchetto) deve essere atomica. Usa `BEGIN TRANSACTION` / `COMMIT` in SQL.
4. **Il bot non si fida dell'Activity:** Il bot deve sempre validare lato server quello che l'Activity dichiara. Questo è il sistema anti-cheat.
5. **Architettura a eventi:** Il sistema di gioco deve essere basato su eventi (`PLAY_CARD`, `ATTACK`, `DICE_ROLL`, `END_TURN`). Questo rende il codice modulare, testabile e facilmente estendibile.
6. **Inizia con 1v1:** Implementa e testa il formato 1v1 prima di passare al Commander multiplayer. La logica di targeting con N giocatori è significativamente più complessa.
7. **Balancing iterativo:** Non cercare di bilanciare le carte prima di giocarle. Implementa un sistema di logging delle partite, gioca partite test, poi aggiusta i numeri. Il bilanciamento viene dal playtest, non

dalla teoria.

8. **Virtualizzazione UI:** Usa la virtualizzazione delle liste nel Deck Builder fin dall'inizio. È molto più facile implementarla subito che aggiungerla dopo quando hai 300 carte.

Documento generato a partire dalla sessione di brainstorming con Gemini AI — Pronto per essere consegnato all'agente di sviluppo.